

TensorFlow equivalent of tutorial_quantum_transfer_learning.py

<https://github.com/PennyLaneAI/qml/issues/28>

ROW 50

TensorFlow equivalent of tutorial_quantum_transfer_learning.py #28

[New issue](#)Closed

rdisipio opened on Jan 13, 2020

Issue description

I'm trying to create a TensorFlow-based equivalent of `tutorial_quantum_transfer_learning.py` but I get an error related to the conversion of PennyLane objects to Tensors.

- **Expected behavior:** Pass values correctly?
- **Actual behavior:** Returns the following error:
ValueError: Attempt to convert a value (<error computing repr(>) with an unsupported type (<class 'pennylane.interfaces.tf.TFQNode.qnode_str'>) to a Tensor.
- **Reproduces how often:** (What percentage of the time does it reproduce?)
I'm not really sure right now what is causing the problem. The stack trace does not point to any line of my code that can create the offence. See below.
- **System information:** (include your operating system, Python version, Sphinx version, etc.)
MacOsX 10.14
TensorFlow 2.0
PennyLane 0.7.0

Source code and tracebacks

```
Training...
Train on 11640 samples, validate on 2910 samples
Epoch 1/20
128/11640 [.....] - ETA: 1:17Traceback (most recent call last):
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
use_multiprocessing=use_multiprocessing)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
total_epochs=epochs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
batch_outs = execution_function(iterator)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
distributed_function(input_fn))
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
per_replica_function, args=(model, x, y, sample_weights))
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/distribute/distribut
return self._extended_call_for_each_replica(fn, args=args, kwargs=kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/distribute/distribut
return self._call_for_each_replica(fn, args, kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/distribute/distribut
return fn(*args, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/autograph/impl/api.py
return func(*args, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
output_loss_metrics=model._output_loss_metrics)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
output_loss_metrics=output_loss_metrics))
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
training=training))
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/trainer
outs = model(inputs, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/base_la
outputs = self.call(cast_inputs, *args, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/network
convert_kwargs_to_constants=base_layer_utils.call_context().saving)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/network
output_tensors = layer(computed_tensors, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/keras/engine/base_la
outputs = self.call(cast_inputs, *args, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/models.py", line 126, in call
q = tf.dtypes.cast(q, tf.float32)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/util/dispatch.py", 1
return target(*args, **kwargs)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/ops/math_ops.py", 11
x = ops.convert_to_tensor(x, name="x")
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/ops.py", 1
return convert_to_tensor_v2(value, dtype, preferred_dtype, name)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/ops.py", 1
as_ref=False)
```

Assignees

josh146

Labels

bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



```
return constant(v, dtype=dtype, name=name)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/constant_op.py", line 100, in constant
    allow_broadcast=True)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/constant_op.py", line 100, in constant
    t = convert_to_eager_tensor(value, ctx, dtype)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/tensorflow_core/python/framework/constant_op.py", line 100, in constant
    return ops.EagerTensor(value, ctx.device_name, dtype)
ValueError: Attempt to convert a value (<error computing repr(>)>) with an unsupported type (<class 'pennylane.'
```

Additional information

It would be nice if you could provide some more examples of interplay between "classical" and quantum ML using TensorFlow instead of (or beside) PyTorch.



josh146 on Jan 13, 2020

Member ...

Hi @rdisipio! Thanks for logging this bug. Have you had a look at our [PennyLane TensorFlow quickstart guide?](#)

It would also be great if you could add a minimal non-working code example that leads to the above traceback (this might be something as simple as the defined QNode and how it is called).



josh146 added **bug** on Jan 13, 2020



josh146 self-assigned this on Jan 13, 2020



rdisipio on Jan 13, 2020

Author ...

Hi,

thanks for the quick reply. The main script is here:

<https://github.com/rdisipio/qnlp/blob/master/train.py>

in turns, this create a model defined in:

<https://github.com/rdisipio/qnlp/blob/master/models.py>

The model is mostly a TF re-implementation of the PyTorch class in the example. As I said, I'm having a hard time to figure out where the offending line is as the traceback never reference any part of my code. I assume I'm passing an object in the wrong format (if I can suggest, an `assert` statement, properly placed, may help to catch such a situation).



rdisipio on Jan 13, 2020

Author ...

I think I'm getting closer to where the issue arises. These lines seem to reproduce the error:

```
q = qml.QNode(self.variational_quantum_circuit(q_in), self.dev).to_tf()
q = tf.convert_to_tensor(q, dtype=tf.float32)
```



where `q_in` is a numpy array like this: `[ 0.08495745 -0.20793235 0.03586568 -0.016984 ]`



rdisipio on Jan 13, 2020 · edited by rdisipio

Edits Author ...

getting there...so the solution was to convert the `QNode` object to a tensor before putting it into the list but also expanding its dimensions, and then concatenate the elements:

```
q_out = []
for q_in in x.numpy():
    q = qml.QNode(self.variational_quantum_circuit(q_in), self.dev).to_tf()
    q = tf.convert_to_tensor(q_in, dtype=tf.float32)
    q = tf.expand_dims(q, 0) #(1,4)
    q_out.append(q)
q_out = tf.concat(q_out, axis=0)
q_out = layers.Dense(self.output_dim)(q_out)
```

But now I get another error:

```
ValueError: No gradients provided for any variable: ['dressed_quantum_circuit/kernel:0'].
```

In the examples, a `tf.GradientTape()` context is created to define the loss and gradient. Is there a known way to make it work with the Keras `fit` API? Or do I have to define a training loop manually? That would be a significant feature to let people just replace a classical layer with a quantum one without having to change large parts of the code.



josh146 on Jan 13, 2020

Member ...

As I said, I'm having a hard time to figure out where the offending line is as the traceback never reference any part of my code. I assume I'm passing an object in the wrong format (if I can suggest, an assert statement, properly placed, may help to catch such a situation).

This is something we'll definitely look into! Unfortunately, TensorFlow makes this quite hard to catch, as their custom gradient decorator wraps and replaces the `QNode`, leading to the very tensorflow heavy tracebacks.

In our current development version of PennyLane (`v0.8-dev`) we have refactored the `QNodes` in such a way that it may solve the issue you are having --- could I get you to try the development branch?

It can be installed via git:

```
pip install git+https://github.com/XanaduAI/pennylane.git#egg=pennylane
```

Once installed, `QNode` is now a constructor function, that now has the same signature as the decorator:

```
q = qml.QNode(self.variational_quantum_circuit(q_in), self.dev, interface="tf")
```



trbromley on Jan 13, 2020

Contributor ...

Hi @rdisipio - I've had a go at making a similar Keras quantum layer before and I think you're on the right track. It took me a bit of playing to get things to work, some things I can quickly spot from your code are:

- Suggest passing `dynamic=True` to the base class `__init__`
<https://github.com/rdisipio/qnlp/blob/39a2a9e5f44441ed9de76642328b893a16ff3a3a/models.py#L67>
- I had some trouble with quantum and classical layers converting between `tf.float32` and `tf.float64` and eventually added `tf.keras.backend.set_floatx('float64')` to any scripts using the layer (although I guess this isn't ideal)



rdisipio on Jan 13, 2020

Author ...

Hi @trbromley ,

thanks for the suggestions. I'm passing the `dynamic` flag before `kwargs` . Also, forcing tensor to be `tf.float64` did not change much. Yeah, it seems to require lots of fiddling to make it work, but it's worth doing so IMO as some pre-trained models are available only for TF.



1

rdisipio on Jan 13, 2020

Author ...

Hi @josh146 ,

thanks for taking care of this. I adapted the line you pointed out to comply with the new API. But now I get:

```
File "/Users/Riccardo/development/qnl/models.py", line 135, in call
  q_out = self.run_quantum_circuit(x)
File "/Users/Riccardo/development/qnl/models.py", line 114, in run_quantum_circuit
  q = qml.QNode(self.variational_quantum_circuit(q_in), self.dev, interface="tf") #0.8-dev
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/decorator.py", line 106, in
  node = PARAMETER_SHIFT_QNODES[model](func, device, mutable=mutable, properties=properties)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/jacobian.py", line 32, in
  super().__init__(func, device, mutable=mutable, properties=properties)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/base.py", line 193, in __i
  _get_signature(self.func)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/base.py", line 70, in _get
  sig = inspect.signature(func)
File "/Users/Riccardo/.pyenv/versions/3.6.9/lib/python3.6/inspect.py", line 3065, in signature
  return Signature.from_callable(obj, follow_wrapped=follow_wrapped)
File "/Users/Riccardo/.pyenv/versions/3.6.9/lib/python3.6/inspect.py", line 2815, in from_callable
  follow_wrapper_chains=follow_wrapped)
File "/Users/Riccardo/.pyenv/versions/3.6.9/lib/python3.6/inspect.py", line 2193, in _signature_from_callabl
  raise TypeError('{!r} is not a callable object'.format(obj))
TypeError: <pennylane.ops.qubit.PauliZ object at 0x148a4d2e8>, <pennylane.ops.qubit.PauliZ object at 0x141f8e
```



josh146 on Jan 13, 2020

Member ...

Ah, this is occurring because the first argument of `QNode` must be the quantum circuit function itself, un-evaluated:

```
q = qml.QNode(self.variational_quantum_circuit, self.dev, interface="tf")
```

I also have a feeling this will fail as `variational_quantum_circuit` is a method, not a function, and the `QNode` was designed to only be used with functions 🤔



rdisipio on Jan 13, 2020 · edited by rdisipio

Edits ... Author ...

Uhm. But then, how do I pass arguments to the function?

```
File "/Users/Riccardo/development/qnl/models.py", line 122, in run_quantum_circuit
  q = qml.QNode(self._variational_quantum_circuit, self.dev, interface="tf") #0.8-dev
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/decorator.py", line 106, in
  node = PARAMETER_SHIFT_QNODES[model](func, device, mutable=mutable, properties=properties)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/jacobian.py", line 32, in
  super().__init__(func, device, mutable=mutable, properties=properties)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/base.py", line 193, in __i
  _get_signature(self.func)
File "/Users/Riccardo/development/qc/lib/python3.6/site-packages/pennylane/qnodes/base.py", line 73, in _get
  func.var_pos = False
AttributeError: 'method' object has no attribute 'var_pos'
```



trbromley on Jan 13, 2020

Contributor ...

Not sure if this helps @rdisipio, I had the following in my `__init__` :

```
def circuit(inputs, parameters):
    qml.templates.embeddings.AngleEmbedding(inputs, wires=list(range(self.units)))
    qml.templates.layers.StronglyEntanglingLayers(parameters, wires=list(range(self.units)))
    return [qml.expval(qml.PauliZ(i)) for i in range(self.units)]

self.dev = qml.device(device, wires=units)
self.layer = qml.QNode(circuit, self.dev, interface="tf")
```

You can then treat `self.layer` as the function you use later on in the `call` method.



rdisipio on Jan 14, 2020 · edited by rdisipio

Edits Author ...

Hi,

thanks for the suggestion. I'm still quite confused. In a derived class from `Keras.layers.Layer`, the parameters are defined in the `build()` function, and the inputs in the `call()` function. So how do I pass these parameters to `circuit(inputs, parameters)` in `__init__` ?

Also, the error I'm still getting

```
ValueError: No gradients provided for any variable: ['dressed_quantum_circuit/kernel:0'].
```

seems to be related to the model's kernel, which I define in `build()` as this:

```
def build(self, input_shape):
    print("Building dressed_quantum_circuit")
    #print(input_shape) #(None, 512)
    self.kernel = self.add_weight(name='kernel',
                                  shape=(self.q_depth, self.n_qubits),
                                  initializer='uniform',
                                  trainable=True)
    super(dressed_quantum_circuit, self).build(input_shape) # Be sure to call this at the end
```

So it's probably not related to `pennylane`, but still, do you have any idea why these parameters do not seem to be picked up by the gradient?



rdisipio on Jan 15, 2020

Author ...

I made some progress. I can get rid of the error concerning missing gradient for kernel variables by defining custom weights rather than calling a Dense layer from `call()`. However, I still get some troubles apparently due to the `dynamic=True` flag. My understanding is that this is needed to manipulate information that does not result in a static graph, such as in RNNs (not sure why that's the case). Is this correct for the `QNode` as well?

If I set `dynamic=False`, I do not know how to loop over the inputs to apply the quantum operations. The tensor object has a `.numpy()` function only if `dynamic=True`.

```
for idx, element in enumerate(w):
    qml.RY(element, wires=idx)
```

This only works if `w` is a numpy array, or I get the error:

```
OperatorNotAllowedInGraphError: iterating over `tf.Tensor` is not allowed in Graph execution. Use Eager mode.
```

On the other hand, if I set `dynamic=True`, the quantum layer does not complain, but I get this other error:

On the other hand, if I set `dynamic=True`, the quantum layer does not complain, but I get this other error:

```
E tensorflow/core/common_runtime/executor.cc:642] Executor failed to create kernel. Invalid argument: Value 0 is not in the range [1, 1]
; NodeDef: {{node Func/_303}}; Op<name=AddN; signature=inputs:N*T -> sum:T; attr=N:int,min=1; attr=T:tf.float32>
[[Func/_303]]
2020-01-15 15:48:31.783444: W tensorflow/core/common_runtime/base_collective_executor.cc:216] BaseCollectiveExecutor:
; NodeDef: {{node Func/_303}}; Op<name=AddN; signature=inputs:N*T -> sum:T; attr=N:int,min=1; attr=T:tf.float32>
[[Func/_303]]
[[gradients/StatefulPartitionedCall_grad/PartitionedCall/gradients/StatefulPartitionedCall_grad/SymbolicOp]]
```



trbromley on Jan 15, 2020

Contributor ...

Hi @rdisipio, let me share with you an example that I got to work:

```
class QuantumLayers(Layer):
    def __init__(
        self,
        units,
        device="default.qubit",
        n_layers=1,
        rotations_initializer="random_uniform",
        rotations_regularizer=None,
        rotations_constraint=None,
        **kwargs,
    ):
        if "input_shape" not in kwargs and "input_dim" in kwargs:
            kwargs["input_shape"] = (kwargs.pop("input_dim"),)
        if "dynamic" in kwargs:
            del kwargs["dynamic"]
        super(QuantumLayers, self).__init__(dynamic=True, **kwargs)

        self.units = units
        self.device = device
        self.n_layers = n_layers
        self.rotations_initializer = initializers.get(rotations_initializer)
        self.rotations_regularizer = regularizers.get(rotations_regularizer)
        self.rotations_constraint = constraints.get(rotations_constraint)

        self.input_spec = InputSpec(min_ndim=2, axes={-1: units})
        self.supports_masking = False

        def circuit(inputs, parameters):
            qml.templates.embeddings.AngleEmbedding(inputs, wires=list(range(self.units)))
            qml.templates.layers.StronglyEntanglingLayers(parameters, wires=list(range(self.units)))
            return [qml.expval(qml.PauliZ(i)) for i in range(self.units)]

        self.dev = qml.device(device, wires=units)
        self.layer = qml.QNode(circuit, self.dev, interface="tf")

    def apply_layer(self, *args):
        return tf.keras.backend.cast_to_floatx(self.layer(*args))

    def build(self, input_shape):
        # assert len(input_shape) == 2
        input_dim = input_shape[-1]
        assert input_dim == self.units

        self.rotations = self.add_weight(
            shape=(self.n_layers, input_dim, 3),
            initializer=self.rotations_initializer,
            name="rotations",
            regularizer=self.rotations_regularizer,
            constraint=self.rotations_constraint,
        )
        self.built = True

    def call(self, inputs):
        return tf.stack([self.apply_layer(i, self.rotations) for i in inputs])

    def compute_output_shape(self, input_shape):
        return input_shape

    def get_config(self):
        config = {
```

Go

```
        "units": self.units,
        "device": self.device,
        "n_layers": self.n_layers,
        "rotations_initializer": initializers.serialize(self.rotations_initializer),
        "rotations_regularizer": regularizers.serialize(self.rotations_regularizer),
        "rotations_constraint": constraints.serialize(self.rotations_constraint),
    }
    base_config = super(QuantumLayers, self).get_config()
    return dict(list(base_config.items()) + list(config.items()))
```

Note that this layer applies the quantum circuit alone - there is no dressing with classical layers so you have to be careful about the input dimension.



rdisipio on Jan 16, 2020

Author

...

Hi **@trbromley**,

thanks for sharing the code. Finally, I can make my code work by following your example, with the main difference that I have to calculate the documents' embeddings outside the network as pass them as input to the network instead of strings. I assume that's a quirk of TensorFlow and has not much to do with PennyLane (to be checked later).

Also, I wonder if the quantum layer you are using is of the same kind of the one that appears in the transfer learning paper (arXiv:1912.08278 references arXiv:1804.00633).



2



josh146 on Jan 17, 2020

Member

...

Closing this for now as the code is now working :)

@rdisipio, if you can write your ML example as a single python script, feel free to make a PR here to add it to the list of demos on <https://pennylane.ai/qml/>!



josh146 closed this as **completed** on Jan 17, 2020

trbromley mentioned this on Mar 2, 2020

[Add Keras layers support PennyLaneAI/pennylane#529](#)

Add Keras layers support #529

<> Code

Merged trbromley merged 84 commits into master from keras_layer on Apr 1, 2020

Conversation 197 Commits 84 Checks 0 Files changed 9

+823 -7



trbromley commented on Mar 2, 2020 · edited

Contributor

This PR provides a `KerasLayer` class for converting a PennyLane `QNode` to a Keras layer, which can then be used within standard Keras features such as the `Model` / `Sequential` models.

Currently, the following is not added:

- A tutorial/demo to show how to use this new feature
- A section in "Using PennyLane" of the documentation showing a higher level picture. This and the above could be added together.
- Supporting input of a `QNodeCollection` instead of a single `QNode`. This should be possible, we need to make the `func.sig` attribute available in the `QNodeCollection` and in `qml.map`.

Providing this functionality would help solve #166 and is also discussed in #133, <https://discuss.pennylane.ai/t/qnode-inside-keras-layer/335/3> and <https://discuss.pennylane.ai/t/creating-and-saving-models/46>.



🔗 Add first commit of keras interface

fe1c10e

🏷️ trbromley added the WIP label on Mar 2, 2020

👁️ trbromley requested a review from josh146 5 years ago

Reviewers

	mariaschuld	✓
	co9oiguy	✓
	josh146	✓

Assignees

trbromley

Labels

enhancement ✨ review-ready 🔥

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet